A Small Language Model for Bengali and Its Application to Grammatical Error Correction

Faria Hasan
ID: 2232830042
Dept. of CSE, North South University
faria.hasan@northsouth.edu

Farhana Rahman
ID: 2233151042
Dept. of CSE, North South University
farhana.rahman04@northsouth.edu

Nirnoy Charisma
ID: 2232287042
Dept. of CSE, North South University
nirnoy.charisma@northsouth.edu

***Abstract*—Although Bengali is used by more than 230 million people, the Bengali language models that are freely available are either very large multilingual models or small encoder-only models which are not suitable for generating free-form text. This report introduces Bangla-SLM-498R, a decoder-only transformer language model with about 17.2 million parameters (14.2 million excluding the embeddings), which was trained from scratch on real Bengali web text, together with a complementary evaluation tool for Bengali Grammatical Error Correction (GEC). The model incorporates Rotary Position Embeddings, RMSNorm, SwiGLU feedforward layers, and tied input and output embeddings in an 8-layer, 6-head architecture. The training data was taken from the `hishab/titulm-bangla-corpus` common-crawl subset, resulting in a final tokenized corpus of about 384 million tokens. Pretraining was carried out on a single NVIDIA RTX 3060 (8 GB) GPU over 5,750 optimizer steps (approximately 1.51 billion tokens, or about four epochs), and the final validation perplexity reached 25.75, having dropped from 8,002 at the start, with no sign of overfitting. The model generates fluent and grammatically correct Bengali text in the style of a newspaper. This report sets out the motivation, architecture, dataset pipeline, training procedure, inference process, evaluation method, and the present state of the project, following the chapter structure of the departmental capstone report template.**

I. Introduction

A. Background and Motivation

Large language models have had a significant impact on natural language processing, although most of the progress has been made in the case of high-resource languages such as English. Despite being one of the most widely spoken languages, Bengali (Bangla) is still not well served. The Bengali-specific models currently available are either very large multilingual encoder-decoder or decoder-only systems that demand a great deal of computational power, or they are small encoder-only models such as BanglaBERT and IndicBERT, which were not designed for generating free-form text [6]. In our related work on a Bengali keyboard app (CSE299), we encountered the same issue: since IndicBERT is an encoder-only model, each grammatical error had to be manually masked before correction, and it produced nonsense when used for text generation. A larger generative model (a Bengali-tuned variant of Qwen3) was also not feasible to run without access to strong GPU resources [**?**]. This gap therefore gives rise to the main question of this project: how much genuinely useful Bengali-language capability can be obtained from a model small enough to be trained from scratch on a single consumer GPU?

B. Purpose and Goal of the Project

The aim of this project is two-fold: first, to design, implement, and carry out a small (in the range of 10 to 20 million parameters) decoder-only transformer language model for Bengali completely from the beginning, by using actual web-crawled text rather than a synthetic or borrowed corpus, so that all the stages of the pipeline – tokenization, data preparation, model architecture, and the training loop itself – are understood and reproducible by the team. Second, the model is to be used as one component of a wider Grammatical Error Correction (GEC) system, with its performance evaluated by comparing it against prompted large language models and simple baseline methods using standard, published GEC metrics rather than ad hoc inspection. The novelty of the project does not consist in beating current large-scale Bengali models, but rather in showing a complete, transparent, and reproducible pipeline for a small language model in a low-resource language, along with a principled evaluation approach for the downstream correction task.

C. Organization of the Report

The report is organised as follows: Section II examines previous work in the areas of transformer architecture design, compute-optimal scaling, and existing Bengali language models. Section III outlines the system design, the software components, and the implementation details for both the pretraining pipeline and the GEC evaluation harness. Section IV contains the results of the current experiments. Section V looks at the wider impact of the project. Section VI gives the project timeline and budget. Section VII investigates the complex engineering problems and activities involved. Section VIII concludes with a summary, along with its limitations and suggestions for future work.

II. Review of the Research Literature

A. Transformer Architecture and Design Decisions

The Transformer architecture [1] is the basis for all modern language models, primarily because it makes use of self-attention instead of recurrence. This project includes a number

of refinements that are now standard in modern, compute-efficient language models, as opposed to the original design from 2017. Rotary Position Embeddings (RoPE) [2] encode information about the position of tokens by rotating the query and key vectors by an angle that is proportional to their position, a method which inherently encodes relative distance and has been found to generalize better than learned absolute position tables, especially when the model is on a small scale. SwiGLU [3], a gated version of the conventional feed-forward block, consistently achieves better performance than plain ReLU-based feed-forward layers when the number of parameters is the same. Instead of using LayerNorm, RMSNorm [4] is employed, eliminating the mean-centering step and resulting in a small but consistent efficiency improvement without any loss of stability.

B. Compute-Optimal Scaling

When the parameter budget is fixed and limited, a significant question in design is determining how much training data is sufficient. Hoffmann et al. [5] found that, for training which is optimal in terms of computing resources, the number of training tokens should increase roughly in line with the number of non-embedding parameters, at a ratio of about 20 to 50 tokens per parameter. This finding directly informed the token budget target used in this project (Section III-D).

C. Current Bengali Language Models

BanglaBERT [6] is an encoder-only Bengali model which was pretrained using a replaced-token-detection objective on 27.5 GB of web-crawled Bengali text and continues to serve as a strong baseline for natural language understanding tasks in Bengali. IndicBERT [7] does the same for the Indic languages, including Bengali, by applying a masked-language-modelling objective across a shared multilingual vocabulary. Since both models are encoder-only, they are not directly suitable for generative correction: a grammatically incorrect sentence cannot simply be completed, it has to be rewritten. This limitation was established in earlier work on a related keyboard project [**?**], in which IndicBERT needed each detected error span to be manually masked and still generated unusable results when asked to produce a fully corrected sentence, which motivated the decoder-only, generation-first approach taken here.

D. The Evaluation of Grammatical Error Correction

Grammatical Error Correction is usually assessed using edit-based precision, recall, and the $F_{0.5}$ score, since this gives more weight to precision than to recall on the basis that unnecessary corrections are more detrimental to a user than errors that are overlooked. GLEU [8] is likewise employed as a fluency-focused measure which penalizes source n-grams that are kept by the system but which the reference corrects. The evaluation framework presented in this project (Section III-F) includes both of these metric families rather than depending only on qualitative inspection.

III. Methodology

A. System Design

The project consists of two successive stages, as shown in Fig. 1: the first stage is a pretraining stage which results in a general-purpose Bengali language model, and the second stage is a GEC evaluation stage that uses a fine-tuned version of that model together with other correction systems for the purpose of benchmarking.

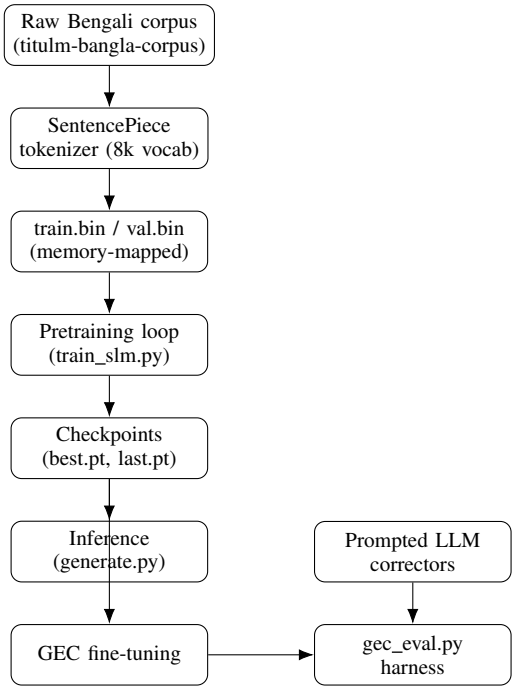

Fig. 1. The end-to-end system pipeline: pretraining (top) produces checkpoints used directly for inference and, after fine-tuning, feeds into the GEC evaluation stage, which also compares against prompted LLM correctors.

B. Software Components

Table I lists the main software tools used in the project.

C. Model Architecture

The `BanglaSLM` model is a decoder-only transformer, summarised in Table II. In each of the 8 transformer blocks, pre-normalisation (RMSNorm) is applied, followed by causal self-attention with RoPE applied to the queries and keys, a residual connection, a second RMSNorm, a SwiGLU feed-forward sublayer, and finally another residual connection. The self-attention mechanism is realised using PyTorch's fused scaled-dot-product-attention kernel. The token embedding table and the output (language-modelling) projection are tied together, which halves the parameter cost linked to the vocabulary.

D. Dataset and Tokenizer

The dataset and tokenizer were built using text from the common-crawl portion of `hishab/titulm-bangla-corpus`, available on the

TABLE I
SOFTWARE TOOLS USED IN THE PROJECT

Tool	Function	Why Selected
PyTorch	Model definition, training loop, autograd	The de facto standard for research-grade deep learning, giving direct low-level control over the training loop
SentencePiece	Subword tokenizer training and encoding	Independent of any specific language; allows training of a vocabulary specific to Bengali rather than reusing an English or multilingual one
Hugging Face Hub	Hosts the prepared dataset; downloaded automatically by `train_slm.py` on its first run	Convenient versioned storage and programmatic download of large binary files, eliminating the need to distribute a multi-gigabyte corpus by hand
NumPy	Memory-mapped binary token storage	Enables training on data larger than available RAM without a heavyweight data-loading framework
NVIDIA RTX 3060 (8 GB)	Local GPU computing for pretraining	Enough VRAM for this model size at the chosen batch settings; avoids reliance on a non-persistent hosted notebook session
Git / GitHub	Version control, branch-based collaboration	Standard collaborative workflow for a multi-person codebase

TABLE II
MODEL CONFIGURATION

Hyperparameter	Value
Vocabulary size	8,000
Context length (block size)	512 tokens
Transformer layers	8
Attention heads	6
Embedding dimension	384
SwiGLU hidden dimension	1,024
Positional encoding	RoPE (base 10,000)
Normalization	RMSNorm (pre-norm)
Total parameters	≈17.2M
Non-embedding parameters	≈14.2M

Hugging Face Hub, comprising real Bengali text gathered from the web rather than artificially generated sentences. In accordance with the compute-optimal scaling advice given in [5], an initial target of about 600 million tokens was set, this figure having been calculated from an early estimate of the parameter count at a rate of roughly 40 to 50 tokens per non-embedding parameter; the number of tokens actually collected and tokenized ended up being 383.6 million (Table III), and the completed training process (Section IV) demonstrated that this amount was still enough to achieve a low and continuously improving validation loss. A SentencePiece byte-pair-encoding tokenizer with an 8,000-token vocabulary and full character coverage was trained on a sample of 200,000 lines of the corpus, together with special tokens for padding, the beginning of a sequence, the end of a sequence, and unknown tokens. The entire corpus was then processed in streaming batches, with an end-of-sequence token added after each line, and was divided deterministically into training and validation sets by means of the CRC32 hash of each line. This hash-based split can be reproduced across different runs without the need to save a random seed or a shuffle order, and the tokenized output is stored as raw `uint16` binary arrays so that the training script can memory-map the data (or, by default, load it entirely into RAM) instead of relying on random-access disk reads during training.

TABLE III
FINAL TOKENIZED DATASET SIZE

Split	Tokens
Training	377,235,487
Validation	6,380,700
Total	383,616,187

E. Training Procedure

The model is trained using AdamW, with weight decay applied only to the two-dimensional (matrix) parameters and not to the normalization or bias terms. The learning rate increases linearly during the first 400 steps until it reaches a maximum of 6×10^{-4} and then decreases according to a cosine decay pattern down to a minimum of 6×10^{-5}. Because of GPU memory limitations, a micro-batch of 32 sequences is used together with 16 steps of gradient accumulation, resulting in an effective batch size of about 262,000 tokens per optimizer update. Automatic mixed precision is used (bfloat16 where available, otherwise float16 with gradient scaling). Checkpoints are written atomically (first to a temporary file, which is then renamed) every 500 steps, and include the model weights, the optimizer state, and the training step, so that the script can automatically resume after an interruption instead of starting over from the beginning. The script deliberately refuses to run at all if a CUDA-capable GPU is not available, on the basis that training a model of this size entirely on a CPU would be far too slow; the run reported in Section IV was performed on a single local NVIDIA RTX 3060 (8 GB) rather than on a hosted notebook GPU.

F. Inference and Text Generation

Generating text from a trained checkpoint is a separate, less demanding process that makes no use of the training script at all. The checkpoint file (`best.pt`, chosen for having the lowest validation loss rather than being the most recent step) is loaded along with its saved configuration dictionary, which is then used to recreate a `BanglaSLM` instance with matching architecture before the saved weights are loaded into it. The SentencePiece tokenizer produced during dataset preparation is loaded separately, since the model works exclusively with integer token IDs and has no understanding of Bengali script. The prompt string is converted into token IDs, passed to the

model's autoregressive `generate` method (which supports temperature and top-k sampling), and the resulting output IDs are decoded back into Bengali text using the same tokenizer. Unlike training, this procedure does not require a GPU: generation can be carried out on an ordinary CPU by loading the checkpoint with `map_location` set to `"cpu"` and omitting the `.cuda()` calls, something that was directly verified during preparation of this report, when the project's actual `best.pt` checkpoint was loaded and a full forward and generation pass was carried out without a GPU present. The only platform-specific detail is that the Windows console does not default to a text encoding that supports Bengali, so the training script and any inference script must explicitly reconfigure the text encoding before printing generated text.

G. Grammatical Error Correction Evaluation Harness

A standalone module, `gec_eval.py`, provides a common evaluation interface so that different correction methods can be fairly compared. The systems implemented or planned comprise: an identity baseline (no correction is made, serving as a lower bound that any real system must exceed); the project's own pretrained SLM, fine-tuned on pairs of incorrect and corrected Bengali sentences; large language model correctors prompted using four different strategies (zero-shot, few-shot, chain-of-thought, and an error-taxonomy-aware prompt covering twelve common categories of Bengali grammatical errors); a self-refine wrapper that critiques and improves an LLM's own output; an edit-level ensemble that combines several systems by majority vote on individual edits rather than on whole sentences; a perplexity-reranking system that selects the most fluent candidate among the outputs of various correctors using the pretrained SLM's own sequence log-probabilities; and a detect-then-correct pipeline that only applies a corrector to sentences identified as erroneous, in order to avoid unnecessary edits. The harness calculates precision, recall, and $F_{0.5}$ at the edit level, GLEU, character-level chrF, the exact-match rate, and an over-correction rate indicating how frequently a system edits a sentence that is already correct.

IV. Investigation/Experiment, Result, Analysis, and Discussion

The pretraining process was carried out until it reached 5,750 optimizer steps. Table IV gives the validation loss and the corresponding perplexity at initialization and at the end of training, while Table V summarises the run's computing characteristics. As a point of reference, a model that has learned no structure at all – one that guesses uniformly among the 8,000 vocabulary tokens – would have a perplexity of 8,000; the fact that perplexity was reduced to 25.75 shows that the model acquired a significant amount of real structure from Bengali text.

The small difference between the final training and validation loss indicates that the model had not overfitted; the validation loss was, in fact, still decreasing at the final evaluation, so a longer run would most likely have reduced the perplexity even further. The run was stopped at 5,750 steps for reporting purposes rather than because performance had plateaued. Qualitative behaviour was examined by generating text from a fixed prompt in the manner described in Section III-F; Fig. 2 displays the model's actual output continuing the Bengali prompt meaning "The Prime Minister of Bangladesh," obtained by loading `best.pt` directly.

TABLE IV
Validation Metrics: Start vs. End of Training

Metric	Start	Final (step 5,750)
Validation loss	8.9874	**3.2483**
Validation perplexity	8,002	**25.75**

TABLE V
Training Run Characteristics

Detail	Value
Optimizer steps	5,750
Tokens per step	262,144 (32 micro-batch × 16 grad-accum × 512)
Total tokens seen	1.51 B (≈4 epochs over 383.6 M tokens)
Wall-clock time	≈7 hours
Throughput	≈60,000 tokens/sec
Final train/val gap	0.056

বাংলাদেশের প্রধানমন্ত্রী দেশবাসীকে শুভেচ্ছা জানিয়েছেন। প্রধানমন্ত্রীর প্রেস সচিব ইহসানুল করিম জানান, প্রধানমন্ত্রী শেখ হাসিনা আজ বিকেলে ... সৌজন্য সাক্ষাৎ করতে পারবেন বলে জানান।

Fig. 2. Actual output of the model in response to the prompt meaning "The Prime Minister of Bangladesh," using a temperature setting of 0.8 and top-k=50 sampling. The output is fluent and grammatically correct Bengali in the style of a newspaper; as is typical of a model with 17 million parameters, its factual content should not be regarded as reliable.

The GEC evaluation harness has so far only been used with its own built-in self-test (a small synthetic benchmark employed to verify the correctness of the metric implementations, for example by confirming that an oracle system gives a value of $F_{0.5} = 1.000$ and that an identity baseline yields 0.000); it has not yet been applied to a real Bengali GEC dataset or to a fine-tuned SLM checkpoint. In accordance with its design, the main role of the pretrained SLM in the harness does not require fine-tuning, since the `PerplexityRerank` system chooses the most fluent candidate among the outputs provided by other, larger correctors based on the model's own length-normalized sequence log-probability, and can therefore be used directly with the checkpoint already trained. A separate `SLMCorrector` pathway, in which the model itself is fine-tuned on pairs of incorrect and correct sentences to carry out correction directly, has not yet been implemented – no fine-tuning script currently exists in the repository. As informal, preliminary evidence of how difficult the correction task is, an earlier evaluation of two off-the-shelf models – IndicBERT and a Bengali-tuned Qwen3 variant – on a 50-sentence Bengali GEC test set found that the encoder-only IndicBERT could not be used for direct generation without manual per-sentence masking, and produced unusable output when compelled to

do so [**?**], supporting the need for the purpose-built pipeline described in this report.

V. Impacts of the Project

A. Impact on Societal and Educational Issues

Bengali is a native language for over 230 million people, yet it is relatively poorly served by open, generative NLP tools. A small, fully understood language model developed from the ground up for Bengali reduces the obstacles for students and small teams who wish to build Bengali-language tools – such as grammar-aware keyboards or writing assistants – without having to rely on large, closed, or English-centric models. This is directly applicable to another keyboard application initiative by the same team [**?**], where a lightweight, locally deployable correction component is preferable to a heavy cloud-hosted LLM because of concerns regarding latency, cost, and offline availability.

B. Impact on Environment and Sustainability

The environmental cost associated with training and deploying billion-parameter language models is well documented, owing to their energy requirements. A model with about 17 million parameters can be pretrained in just a few hours (approximately 7 hours in this project) on a single consumer-grade GPU and can perform inference on ordinary consumer hardware, including CPUs, as directly confirmed in Section III-F. Where such a model meets the accuracy requirements of a particular task, such as Bengali grammar correction, using it instead of a much larger general-purpose model results in a considerably smaller compute and energy footprint per inference.

VI. Project Planning and Budget

Table VI gives a summary of the project's phased timeline. Since the project involves software and research rather than hardware construction, the relevant budget (Table VII) is mainly made up of compute and storage resources and is only an approximation.

TABLE VI
Project Phases

Phase	Status
Architecture design and code review (`slm_model.py`)	Complete
Dataset sourcing and tokenizer training	Complete
Corpus tokenization (train.bin/val.bin)	Complete
Pretraining (5,750 steps, RTX 3060)	Complete
Inference / text generation from checkpoint	Complete
GEC evaluation harness implementation	Complete (self-test only)
Bengali GEC benchmark dataset (incorrect/correct pairs)	Planned
GEC fine-tuning of pretrained checkpoint	Planned
Full GEC benchmark across all systems	Planned
Final report and viva preparation	In progress

TABLE VII
Approximate Resource Budget

Resource	Approx. Cost	Notes
Compute (GPU-hours)	\$0	Local NVIDIA RTX 3060, $\approx$7 hours
Dataset storage (HF Hub)	\$0	Free-tier public dataset repo
Checkpoint hosting	\$0	Not yet on a public repo; shared directly between teammates as compressed files

VII. Complex Engineering Problems and Activities

A. Complex Engineering Problems (CEP)

The complex engineering problem attributes addressed by this project are set out in Table VIII.

TABLE VIII
Complex Engineering Problem Attributes

Attribute	Application to This Project
Depth of knowledge	Requires transformer architecture theory, linear algebra, probability and information theory (cross-entropy, perplexity), and optimization (WK3–WK6)
Range of conflicting requirements	Model size, dataset size, and available (free-tier) compute trade off directly against one another
Depth of analysis	Choice of positional encoding, normalization scheme, and vocabulary size each required comparing multiple established alternatives
Familiarity of issues	Memory-mapped large-file data loading and mixed-precision numerical stability are non-obvious, less commonly taught issues
Extent of applicable codes	No formal standard exists for small-language-model design; best practice is drawn from recent research literature
Stakeholder involvement	Course faculty, project teammates, and prospective downstream users of a Bengali correction tool
Interdependence	Tokenizer, dataset pipeline, model code, and evaluation harness are tightly coupled subsystems that must remain mutually consistent

VIII. Conclusions

A. Summary

This report outlines Bangla-SLM-498R, a small decoder-only transformer language model for Bengali which was pretrained from scratch using real web-crawled text, along with a supporting evaluation tool for Bengali Grammatical Error Correction. The model's architecture incorporates RoPE, RMSNorm, SwiGLU, and tied embeddings, giving it a total of about 17.2 million parameters. Pretraining was carried out on a single consumer-grade GPU and took approximately seven hours, consisting of 5,750 optimizer steps (corresponding to

about four epochs over a corpus of 383.6 million tokens), reaching a final validation perplexity of 25.75, a significant reduction from the initial value of 8,002, with no indication of overfitting. The trained checkpoint generates fluent, grammatical Bengali text, confirmed both by the qualitative sample in Fig. 2 and by direct, successful loading and generation with the checkpoint during preparation of this report.

B. Limitations

The project has several current limitations. The final tokenized corpus (383.6 million tokens) came in below the original 600-million-token target, and while the observed train/val gap suggests the model would likely have continued improving with more data or more steps, this was not tested further within the current run. The GEC evaluation harness has only been validated against its own synthetic self-test and has not yet been run against a real Bengali GEC dataset or a fine-tuned checkpoint of the project's own model; correspondingly, no quantitative $F_{0.5}$, GLEU, or chrF results yet exist for the SLM on the correction task itself, only for the harness's internal self-test. The `SLMCorrector` fine-tuning path described in the harness design has no implementation yet. Checkpoint distribution is also currently informal – trained checkpoints are shared directly between teammates as compressed archives rather than through a versioned model repository, and PyTorch's zip-based checkpoint format requires care when compressing and decompressing it a second time, since naively re-zipping an already-extracted checkpoint directory does not reproduce a directly loadable file. Finally, being a small model by design, Bangla-SLM-498R is not expected to match large multilingual or Bengali-specific LLMs on open-ended generation quality, long-range coherence, or factual reliability; its intended niche is lightweight, well-understood, and cheaply deployable correction-oriented use, particularly as a fluency re-ranker rather than a direct corrector.

C. Future Improvement

Planned future work includes: assembling a labeled Bengali GEC benchmark dataset of incorrect/correct sentence pairs; running the `PerplexityRerank` system – usable immediately with the existing pretrained checkpoint – against candidates from prompted LLM correctors, and reporting quantitative $F_{0.5}$, GLEU, and chrF results; implementing the fine-tuning script needed for the `SLMCorrector` path and comparing it against the reranking approach; and, longer-term, integrating the resulting correction system into the related Bengali keyboard application [**?**] as a lightweight, locally deployable alternative to cloud-hosted large language models.

REFERENCES

[1] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[2] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[3] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[4] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[5] J. Hoffmann *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[6] A. Bhattacharjee *et al.*, "BanglaBERT: Language model pretraining and benchmarks for low-resource language understanding evaluation in Bangla," in *Findings of the Association for Computational Linguistics: NAACL*, 2022.

[7] [Online]. Available: https://huggingface.co/ai4bharat/indic-bert

[8] C. Napoles, K. Sakaguchi, M. Post, and J. Tetreault, "Ground truth for grammatical error correction metrics," in *Proc. 53rd Annual Meeting of the Association for Computational Linguistics*, 2015.

[9] [Online]. Available: https://huggingface.co/datasets/hishab/titulm-bangla-corpus

[10] A. Karpathy, "nanoGPT." [Online]. Available: https://github.com/karpathy/nanoGPT